

作为最后批注，我必须指出，`createInvestment` 返回的“未加工指针”（raw pointer）简直是对资源泄漏的一个死亡邀约，因为调用者极易在这个指针身上忘记调用 `delete`。（即使他们使用 `auto_ptr` 或 `tr1::shared_ptr` 来执行 `delete`，他们首先必须记得将 `createInvestment` 的返回值存储于智能指针对象内。）为与此问题搏斗，首先需要对 `createInvestment` 进行接口修改，那是条款 18 面对的事。

### 请记住

- 为防止资源泄漏，请使用 RAII 对象，它们在构造函数中获得资源并在析构函数中释放资源。
- 两个常被使用的 RAII classes 分别是 `tr1::shared_ptr` 和 `auto_ptr`。前者通常是较佳选择，因为其 *copy* 行为比较直观。若选择 `auto_ptr`，复制动作会使它（被复制物）指向 `null`。

## 条款 14：在资源管理类中小心 *coping* 行为

Think carefully about copying behavior in resource-managing classes.

条款 13 导入这样的观念：“资源取得时机便是初始化时机”（*Resource Acquisition Is Initialization*; RAII），并以此作为“资源管理类”的脊柱，也描述了 `auto_ptr` 和 `tr1::shared_ptr` 如何将这个观念表现在 heap-based 资源上。然而并非所有资源都是 heap-based，对那种资源而言，像 `auto_ptr` 和 `tr1::shared_ptr` 这样的智能指针往往不适合作为资源掌管者（resource handlers）。既然如此，有可能偶而你会发现，你需要建立自己的资源管理类。

例如，假设我们使用 C API 函数处理类型为 `Mutex` 的互斥器对象（mutex objects），共有 `lock` 和 `unlock` 两函数可用：

```
void lock (Mutex* pm);           //锁定 pm 所指的互斥器。
void unlock (Mutex* pm);        //将互斥器解除锁定。
```

为确保绝不会忘记将一个被锁住的 `Mutex` 解锁，你可能会希望建立一个 class 用来管理机锁。这样的 class 的基本结构由 RAII 守则支配，也就是“资源在构造期间获得，在析构期间释放”：

```
class Lock {
public:
    explicit Lock (Mutex* pm)
        : mutexPtr (pm)
    { lock (mutexPtr); }           //获得资源
```

```

    ~Lock() { unlock(mutexPtr); }    //释放资源
private:
    Mutex *mutexPtr;
};

```

客户对 Lock 的用法符合 RAII 方式:

```

Mutex m;    //定义你需要的互斥器
...
{
    //建立一个区块用来定义 critical section.
    Lock ml(&m); //锁定互斥器.
    ...        //执行 critical section 内的操作.
}              //在区块最末尾, 自动解除互斥器锁定.

```

这很好, 但如果 Lock 对象被复制, 会发生什么事?

```

Lock ml1(&m);    //锁定 m
Lock ml2(ml1);   //将 ml1 复制到 ml2 身上。这会发生什么事?

```

这是某个一般化问题的特定例子。那个一般化问题是每一位 RAII class 作者一定需要面对的: “当一个 RAII 对象被复制, 会发生什么事?” 大多数时候你会选择以下两种可能:

- **禁止复制。**许多时候允许 RAII 对象被复制并不合理。对一个像 Lock 这样的 class 这是有可能的, 因为很少能够合理拥有“同步化基础器物”(synchronization primitives)的复件(副本)。如果复制动作对 RAII class 并不合理, 你便应该禁止之。条款 6 告诉你怎么做: 将 *copying* 操作声明为 private。对 Lock 而言看起来是这样:

```

class Lock: private Uncopyable {    //禁止复制。见条款 6.
public:
    ...
};

```

- **对底层资源祭出“引用计数法”(reference-count)。**有时候我们希望保有资源, 直到它的最后一个使用者(某对象)被销毁。这种情况下复制 RAII 对象时, 应该将资源的“被引用数”递增。tr1::shared\_ptr 便是如此。

通常只要内含一个 tr1::shared\_ptr 成员变量, RAII classes 便可实现出 reference-counting *copying* 行为。如果前述的 Lock 打算使用 reference counting, 它可以改变 mutexPtr 的类型, 将它从 Mutex\* 改为 tr1::shared\_ptr<Mutex>。然而很不幸 tr1::shared\_ptr 的缺省行为是“当引用次数为 0 时删除其所指物”, 那不是我们所要的行为。当我们用上一个 Mutex, 我们想要做的释放动作是解

除锁定而非删除。

幸运的是 `tr1::shared_ptr` 允许指定所谓的“删除器”（`deleter`），那是一个函数或函数对象（`function object`），当引用次数为 0 时便被调用（此机能并不存在于 `auto_ptr`——它总是将其指针删除）。删除器对 `tr1::shared_ptr` 构造函数而言是可有可无的第二参数，所以代码看起来像这样：

```
class Lock {
public:
    explicit Lock(Mutex* pm)           //以某个 Mutex 初始化 shared_ptr
    : mutexPtr(pm, unlock)             // 并以 unlock 函数为删除器.
    {
        lock(mutexPtr.get());         //条款 15 谈到 "get"
    }
private:
    std::tr1::shared_ptr<Mutex> mutexPtr; //使用 shared_ptr
};                                         //替换 raw pointer
```

请注意，本例的 `Lock class` 不再声明析构函数。因为没有必要。条款 5 说过，`class` 析构函数（无论是编译器生成的，或用户自定的）会自动调用其 `non-static` 成员变量（本例为 `mutexPtr`）的析构函数。而 `mutexPtr` 的析构函数会在互斥器的引用次数为 0 时自动调用 `tr1::shared_ptr` 的删除器（本例为 `unlock`）。（当你阅读这个 `class` 的原始码，或许会感谢其中有一条注释指出：你并没有忘记析构，你只是倚赖了编译器生成的缺省行为。）

- **复制底部资源。**有时候，只要你喜欢，可以针对一份资源拥有其任意数量的复件（副本）。而你需要“资源管理类”的唯一理由是，当你不再需要某个复件时确保它被释放。在此情况下复制资源管理对象，应该同时也复制其所包覆的资源。也就是说，复制资源管理对象时，进行的是“深度拷贝”。

某些标准字符串类型是由“指向 `heap` 内存”之指针构成（那内存被用来存放字符串的组成字符）。这种字符串对象内含一个指针指向一块 `heap` 内存。当这样一个字符串对象被复制，不论指针或其所指内存都会被制作出一个复件。这样的字符串展现深度复制（`deep copying`）行为。

- **转移底部资源的拥有权。**某些罕见场合下你可能希望确保永远只有一个 `RAII` 对象指向一个未加工资源（`raw resource`），即使 `RAII` 对象被复制依然如此。此时资源的拥有权会从被复制物转移到目标物。一如条款 13 所述，这是 `auto_ptr` 奉行的复制意义。

*Coping* 函数（包括 *copy* 构造函数和 *copy assignment* 操作符）有可能被编译器自动创建出来，因此除非编译器所生版本做了你想要做的事（条款 5 提过其缺省行为），否则你得自己编写它们。某些情况下你或许也想支持这些函数的一般版本，这样的版本描述于条款 45。

### 请记住

- 复制 RAII 对象必须一并复制它所管理的资源，所以资源的 *copying* 行为决定 RAII 对象的 *copying* 行为。
- 普遍而常见的 RAII class *copying* 行为是：抑制 *copying*、施行引用计数法（reference counting）。不过其他行为也都可能被实现。

## 条款 15: 在资源管理类中提供对原始资源的访问

Provide access to raw resources in resource-managing classes.

资源管理类（resource-managing classes）很棒。它们是你对抗资源泄漏的堡垒。排除此等泄漏是良好设计系统的根本性质。在一个完美世界中你将倚赖这样的 classes 来处理和资源之间的所有互动，而不是玷污双手直接处理原始资源（raw resources）。但这个世界并不完美。许多 APIs 直接指涉资源，所以除非你发誓（这其实是一种少有实际价值的举动）永不录用这样的 APIs，否则只得绕过资源管理对象（resource-managing objects）直接访问原始资源（raw resources）。

举个例子，条款 13 导入一个观念：使用智能指针如 `auto_ptr` 或 `tr1::shared_ptr` 保存 **factory** 函数如 `createInvestment` 的调用结果：

```
std::tr1::shared_ptr<Investment> pInv(createInvestment()); //见条款 13
```

假设你希望以某个函数处理 `Investment` 对象，像这样：

```
int daysHeld(const Investment* pi); //返回投资天数
```